04 / GUÍA DE PSEUDOCÓDIGO

Del modelo geométrico al algoritmo

EX30 · Planificación antes de programar

Objetivo. Convertir una necesidad física en un procedimiento numérico ordenado, antes de escribir Python. Las comprobaciones sirven para detectar un error de signo, de eje o de orden de multiplicación.

1. Del problema físico a las entradas y salidas

Antes de calcular, identificar qué cambia y qué permanece fijo.

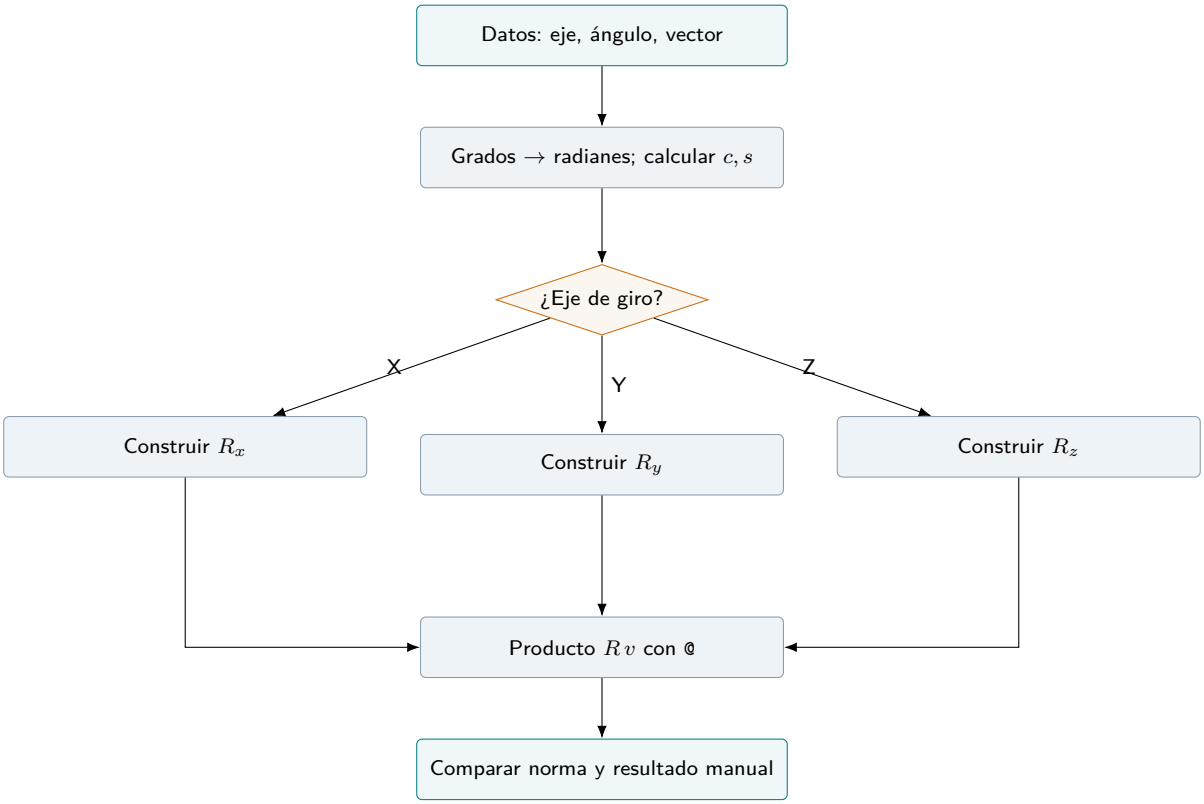
1. **Operación:** ¿se gira un vector físico o se cambian los ejes que lo describen?
2. **Entradas:** eje de giro (X, Y o Z), ángulo en grados, vector de tres componentes y, si procede, brazo de palanca.
3. **Salida:** vector rotado o sus componentes en otro marco, norma, matriz compuesta o momento.
4. **Control:** resultado manual, norma, ortogonalidad y transformación inversa.

2. Del álgebra al algoritmo: construir una rotación 3D

1. Leer el ángulo θ y el eje elegido.
2. Convertir θ de grados a radianes.
3. Calcular $c = \cos\theta$ y $s = \sin\theta$.
4. Según el eje, colocar el 1 sobre su componente invariante y formar el bloque de rotación 2D en las otras dos.
5. Comprobar que la matriz construida tiene forma `(3,3)`.
6. Aplicar el producto matriz-vector y comparar con el resultado manual.

No almacenar una matriz para un ángulo y utilizarla después de cambiar la variable de ángulo: reconstruirla.

3. Diagrama visual de decisiones



4. Pseudocódigo guiado — una rotación activa

```
INICIO
    elegir eje y angulo_en_grados
    convertir angulo a radianes
    calcular coseno y seno
    construir matriz R para ese eje
    definir vector inicial v
    calcular v_rotado = R @ v
    comprobar forma, resultado esperado y norma
FIN
```

Aquí no se decide aún cómo estructurar variables ni funciones; lo importante es seguir el flujo matemático.

5. Pseudocódigo — dos giros con orden explícito

```
INICIO
    construir R_primera para el primer eje y angulo
    construir R_segunda para el segundo eje y angulo
    calcular intermedio = R_primera @ vector
    calcular final = R_segunda @ intermedio
    repetir invirtiendo el orden de los dos giros
    comparar ambos vectores (allclose)
    comprobar normas
FIN
```

Trampa que se quiere detectar: cambiar el orden de multiplicación de las matrices sin darse cuenta. En 3D, los resultados pueden ser diferentes aunque ambas normas se conserven.

6. Pseudocódigo — ingeniería: fuerza y momento

1. Definir la fuerza inicial en newtons y construir las dos matrices de giro requeridas.
2. Aplicar las rotaciones en el orden físico enunciado; comprobar la magnitud.
3. Definir el brazo en metros y obtener el momento con producto vectorial (EX27).
4. Mostrar fuerza final (N) y momento (N m) con interpretación de signos.

7. Pseudocódigo — cambio de referencia pasivo

1. Escribir las componentes globales de la fuerza física fija.
2. Construir la matriz que expresa los ejes locales en los globales.
3. Calcular componentes locales mediante `R.T @ F_global`.
4. Reconstruir las globales con `R @ F_local` y validar el vector completo.
5. Comprobar la conservación de la norma, sin confundirla con igualdad de componentes.

8. Autocomprobación antes de programar

1. ¿Sé qué componente debe mantenerse igual para cada eje de giro?
2. ¿Los signos de las tres matrices coinciden con la regla de la mano derecha?
3. ¿Sé explicar por qué `R2 @ R1 @ v` aplica `R1` primero?
4. ¿Uso `np.allclose` para comparar resultados numéricos que pueden contener errores de redondeo?
5. ¿Distinguiré fuerza física girada y componentes de fuerza en un marco local girado?

Idea final: construir → aplicar → comprobar → interpretar. En este ejercicio es más importante dominar los ejes y el orden que reducir líneas de código.